

《人工智能通识》

第二讲：走进大模型——你必须懂的关键词

周天航

重质油全国重点实验室
中国石油大学（北京）未来能源学院

2026 年 9 月



先回顾：上一讲留下的三个问题，你还能用大白话回答吗？

Q1 它是什么？ 模型是引擎，产品是汽车

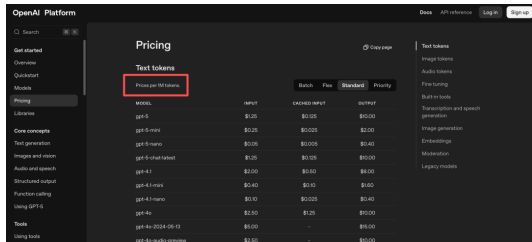
Q2 它怎么学会的？ 预测下一个词，自监督，参数收敛

Q3 它怎么工作的？ 多层堆叠 + 注意力 + 逐词生成

上一讲还埋了三个钩子：**Tokenizer**、**Embedding**、**上下文长度**。

今天把它们，连同你在用大模型时一定会碰到的另外四个词，**一次讲清楚**。

情境：打开一家大模型公司的定价页面



Pricing		Batch		File	Standard	Priority
Text tokens		cached input		output		
Prices per 1M tokens						
Models	input					
gpt-5	\$1.25			\$0.025		\$10.00
gpt-5-mini	\$0.25			\$0.025		\$2.00
Text generation						
gpt-5-mono	\$0.05			\$0.005		\$0.40
Images and vision						
gpt-5-chat-latest	\$1.25			\$0.025		\$10.00
Audio and speech						
gpt-4.1	\$3.00			\$0.60		\$8.00
Structured output						
gpt-4.1-mini	\$0.40			\$0.10		\$1.60
Function calling						
gpt-4.1-mono	\$0.10			\$0.025		\$0.40
Using GPT-4						
gpt-4o	\$2.50			\$1.25		\$10.00
Tools						
gpt-4o-2024-08-13	\$5.00			-		\$16.00
Using tools						
gpt-4o-audio-preview	\$1.50			-		\$10.00

价格单位是“每 100 万 tokens”，输入和输出还分开计价，文本、图像、音频各有各的价。

Q:你手机里的 ChatGPT、豆包是按月订阅的，为什么 API 却按 token 计费？

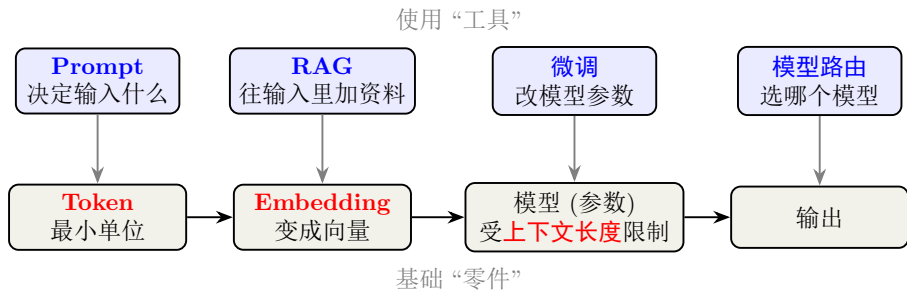
Q:token 到底是什么？为什么它能代表“用了多少”？

国内可直接打开的例子：
[DeepSeek 模型定价页](#)

课堂互动①·猜一猜

一篇 1 万字的中文文献，大概是多少个 token？让模型读一遍要花多少钱？先凭直觉写个数，本讲结束后我们算一遍。

本讲七个关键词, 一张图看位置



Token、Embedding、上下文长度是模型内部的“零件”；
Prompt、RAG、微调、模型路由是把模型用起来的“工具”。

本讲学习目标

知识目标

说清 Token、Embedding、上下文长度三个基础概念; 解释大模型为什么按 token 计费; 知道 Prompt、RAG、微调、模型路由各解决什么问题。

能力目标

能估算一段文本的 token 数量与调用成本; 面对一个具体需求, 能判断先用 Prompt、RAG 还是微调。

素养目标

建立使用 AI 的成本意识与效率意识; 理解“术语”背后的工程逻辑, 不被名词吓住。

学习建议: 每个词都先看“它解决什么问题”, 再看“它是什么”。

第一部分 · Token (词元): 语言的最小计量单位

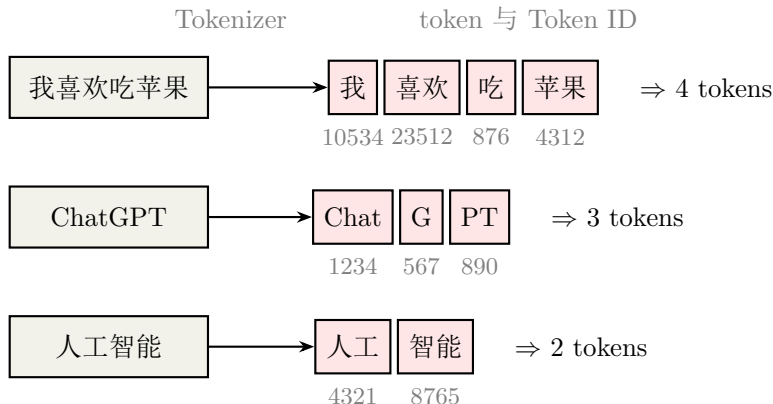
Token 是大模型**理解和处理文本的基本单位**

直观地说, 可以先把它当成 “一个词”。

你输入一句自然语言, 模型第一步**不是**直接处理原始字符, 而是先由 **Tokenizer (分词器)** 把文本切成一个个 token。

分词 (tokenization) 是文本处理的第一步, 它决定了后续 embedding 和模型理解的基本单元。

Tokenizer 是怎么切的？三个例子



注意 “ChatGPT” 被切成了三段：词表里没有完整的 “ChatGPT” 这一条，只能拆成更小的**子词**。

示例中的 Token ID 为示意，不同模型的词表不同，编号也不同。

中文和英文, 切法不一样

中文

- ▶ 通常一个汉字或一个词算一个 token
- ▶ 也可能按分词规则切成更小的单位
- ▶ 粗略估计: 1 个汉字 $\approx 1 \sim 1.5$ 个 token

英文

- ▶ 一个 token 不一定是完整单词
- ▶ 可能是单词的一部分, 甚至一个字符
- ▶ 粗略估计: 1 个单词 ≈ 1.3 个 token

英文比中文更容易被拆: 长单词、专业术语、生造词, 往往一个词变好几个 token。

Q:同一句话翻译成中英文, 哪个更“省 token”? 为什么这对写 Prompt 有影响?

数一数

课堂互动②

“中国石油大学 (北京) 未来能源学院” 这 16 个字, 你觉得会被切成几个 token?

- A. 4 ~ 6 个 B. 8 ~ 12 个 C. 16 个 D. 超过 20 个

玩法: 先举手选, 再现场打开在线 tokenizer 输入这句话, 看实际切分结果。

推荐 token4all.cn/token (国内可直接打开, 免登录, 会把每个 token 标出来);
备用 llmabacus.com/estimate。

再试三句:

- ▶ “催化裂化”
- ▶ “Fluid Catalytic Cracking”
- ▶ “FCC”

专业词会被拆成几段?
英文术语呢?
缩写最省吗?

结论先放这里: 常见的词省 token, 生僻的词费 token。

Token ID 与词表 (Vocabulary)

Tokenizer 输出的不是文字, 而是一串**整数编号** ——Token ID。

固定的 ID 不是随机生成的, 由模型的**词表**事先定义好

唯一的 每个 token 在词表里有且只有一个编号

有用的 每个 ID 直接对应 embedding 表中的一行 (下一部分讲)

词表在**训练模型之前**就定好了; 词表有多大, embedding 表就有多少行。

类比: 词表就像一本字典的目录, Token ID 是页码。

两种付费方式: 产品订阅 vs. API 按量计费

	产品 (如 ChatGPT App)	API (开发者调用)
怎么收费	按月订阅 / 套餐	按实际消耗的 token 精确计费
计量单位	月	每 100 万 tokens (1M tokens)
区分类型	不区分	文本 / 图像 / 音频 token 价格不同
输入输出	不区分	输入与输出 token 分开定价, 输出通常更贵

回到第一讲的判断标准: 按 token 计费的, 多半是在直接用“模型”。

算一笔账：让模型读一篇文献要多少钱？

一篇中文文献	约 1 万字 $\approx$ 1.2 万 token (输入)
让它写 500 字摘要	约 600 token (输出)
假设价格	输入 2 元 / 1M tokens, 输出 8 元 / 1M tokens
成本	$0.012 \times 2 + 0.0006 \times 8 \approx$ 0.03 元

读 1000 篇文献做综述，也只要 30 元左右——比一次实验的试剂便宜得多。

价格为示意，各家模型差异很大；但数量级是对的。

Q:回到开场的猜数：你猜对数量级了吗？哪一步最容易估错？

化工同学的类比: token 就是“计量单位”

买原料	按吨计	不管你买来干什么
做反应	按摩尔算	反应方程式只认物质的量
用大模型	按token计	模型只认 token, 不认“字”或“句”

所以, 关于 Token 记住三句话:

1. 它是模型处理文本的**最小单位**, 由 Tokenizer 切出来;
2. 每个 token 有一个固定的**整数编号** (Token ID);
3. 它是**计费**和**上下文长度**的计量单位。

Q:编号只是一个数字, 模型怎么从数字里“读出”意思? $\Rightarrow$ Embedding

第二部分 · Embedding (嵌入): 把 token 变成向量

Embedding 就是根据 Token ID, 把每个 token 变成一串数字

这串数字叫向量 (vector), 长度叫维度 (dimension)。

OpenAI text-embedding-3-small 1536 维

OpenAI text-embedding-3-large 3072 维

也就是说, 一个词在模型眼里, 是一个有一两千个坐标的点。

Embedding 表: 一个 $N \times M$ 的矩阵

Embedding 表 (矩阵 $N \times M$)

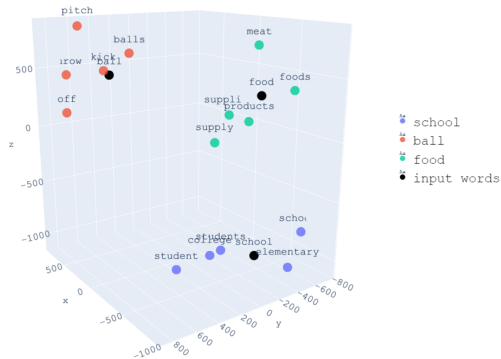
ID 0	[0.01, -0.02, 0.03, ...]	N 行 = 词表大小
ID 1	[0.12, 0.50, -0.20, ...]	
ID 10348 (“今天”)	[0.70, -0.10, 0.90, ...] = v_1	
ID 3598 (“天气”)	[-0.20, 0.30, 0.80, ...] = v_2	
	$\vdots$	
ID $N - 1$	[...]	

M 维 (如 1536 / 3072)

N 是词表大小 (有多少个 token), M 是每个 token 的向量维度。

Token ID 就是**行号**: 查表取出对应的那一行, 就是这个 token 的向量。

这些数字不是随便填的：语义相近，向量就相近



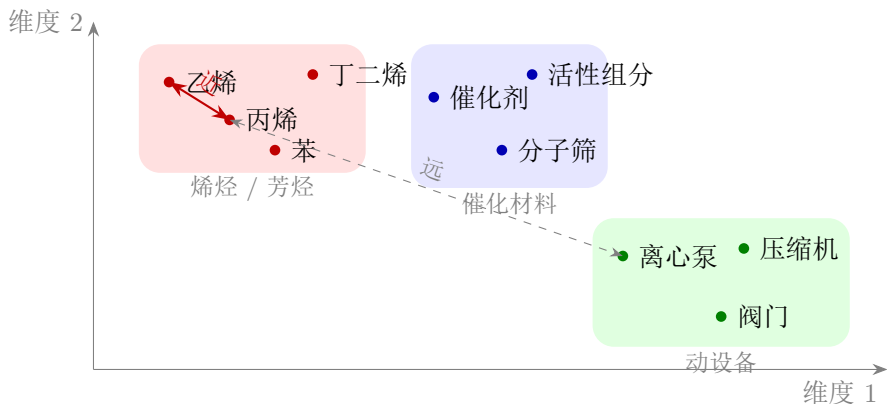
向量是**训练**出来的：同一类词的向量相似度高，不同类的词相似度低。

图中 school、student、college 挤在一起，food、meat、supply 挤在一起，形成清晰的**语义簇**。

维度 M 越大，向量能表达的语义特征越丰富，越能区分意思相近的词。

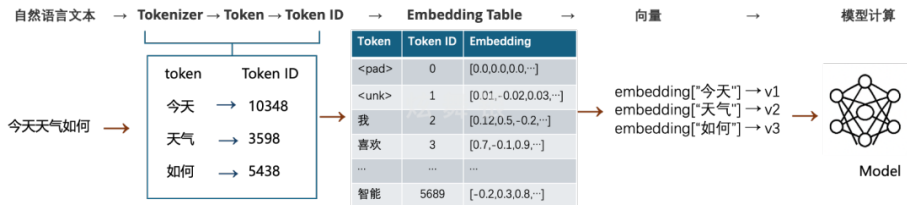
t-SNE 把高维向量压到三维后的可视化

用二维平面感受一下 (真实向量有上千维)



“距离”就是“意思差多少”：乙烯和丙烯挨得近，离“离心泵”很远。
后面讲 RAG 时，“找最相关的文档”靠的正是**算向量之间的距离**。

串起来: 从一句话到模型能算的数字



“今天天气如何” → Tokenizer 切成 [今天, 天气, 如何] → 查词表得到 ID [10348, 3598, 5438]

→ 按 ID 到 Embedding 表取出 v_1, v_2, v_3 → 送进模型计算。

Token ID 和 Embedding 的行是一一对应的: 词表定好了, embedding 表就按同样顺序初始化。

小组：给专业词“画坐标”

课堂互动③

8 个词：乙烯、丙烯、催化剂、分子筛、离心泵、阀门、精馏塔、反应器。
4 人一组，在草稿纸上画一个二维平面，把 8 个词放上去，意思越近的放得越近。3 分钟。

然后回答：

1. 你们用了哪两个“坐标轴”？（如：物质 $\leftrightarrow$ 设备，静设备 $\leftrightarrow$ 动设备）
2. 不同组画出来一样吗？为什么模型学出来的坐标轴，人往往说不清是什么？

模型的 embedding 就是几千个这样的坐标轴，只不过是它自己从数据里学出来的。

化工同学的类比：向量就是一张“物性参数表”

物质	沸点	密度	极性	分子量
乙醇	78	0.79	强	46
甲醇	65	0.79	强	32
正己烷	69	0.66	弱	86

用一组数字描述一种物质，数字接近的物质性质相似——乙醇和甲醇“挨得近”。

Embedding 做的是同一件事：用 1536 个“参数”描述一个词，只是这些参数不是人定义的，而是模型从海量文本里自己学出来的。

Q:有了向量，模型一次能“看”多少个向量？ $\Rightarrow$ 上下文长度

上下文长度 = 模型一次推理中能“看到”的连续 token 数量

第三部分 · 上下文长度 (Context Length): 模型的“记忆窗口”

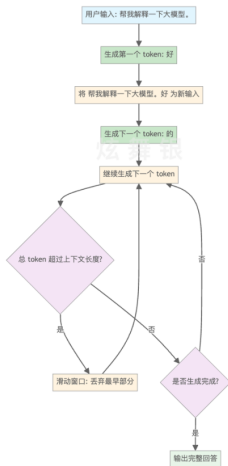
它决定了模型在生成下一个 token 时, 能参考多少之前的文本。

上下文越长 能读更长的文档, 多轮对话更连贯

上下文越短 读到一半就“忘了”开头

单位仍然是 token ——这就是为什么 Token 要放在第一个讲。

生成时, 上下文是怎么滚动的?



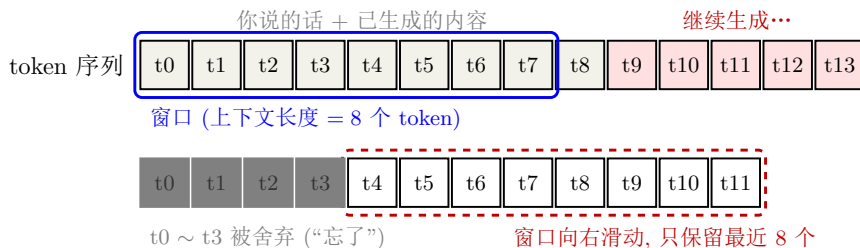
你输入“帮我解释一下大模型。”

1. 模型生成第一个 token, 比如“好”;
2. 把原始输入 + 已生成的“好”作为新输入, 生成下一个“的”;
3. 如此循环, 直到生成完整回答。

每一步模型参考的都是: 你说的内容 + 它自己已经生成的内容。

这两部分加起来, 不能超过上下文长度。

超出窗口会怎样？滑动窗口：忘掉最前面的



输入加上已生成的内容一旦超过上下文长度，
模型就把**最前面超出的部分舍弃**，只保留最近的一段继续生成。

这就是为什么聊得太久，它会“忘记”你一开始说过什么。

各模型的“记忆窗口”有多大？

模型	上下文长度	大约相当于
GPT-3	2K token	一篇 3 页的短文
GPT-4	8K / 32K	一篇长论文 / 一章教材
GPT-4 Turbo	128K	一本 200 页的书
GPT-4o	200K	一本厚书
DeepSeek-R1	128K	一本 200 页的书

K = 1000 个 token。数字随版本更新变化很快，请以官网为准。

从 2K 到 200K，三年涨了 100 倍——这是大模型“读长文”能力突飞猛进的直接原因。

判断题 (先举手, 再揭晓)

课堂互动④

1. 上下文长度越长, 模型的回答就一定越好。
 2. 你和 ChatGPT 聊了两小时后它“忘事”, 是因为它的参数被改掉了。
 3. 把一篇 300 页的手册整个贴给 8K 上下文的模型, 它会读完再回答。
-
1. **✗** 不一定。更长的上下文意味着更高的成本, 而且塞进太多无关内容会**稀释注意力**。
 2. **✗** 参数在推理时不变。忘事是因为早期对话被**滑出了窗口**。
 3. **✗** 超出的部分会被截断, 它只“看到”了一部分——这正是下一部分 RAG 要解决的问题。

三个基础词, 一句话串起来

文本被 Tokenizer 切成 **token**, 每个 token 查表变成 **embedding** 向量,
模型一次最多读 **上下文长度** 那么多个 token, 逐个预测下一个。

Token	计量单位	计费、上下文都按它算
Embedding	数字表示	意思相近, 向量相近
上下文长度	记忆窗口	超出就忘掉最前面

Q:为什么同样是 1000 个 token, 有的模型贵、有的模型便宜? $\Rightarrow$ 参数与计费原理

第四部分 · 参数与计费：为什么按 token 收费？

回顾第一讲：模型可以类比为 $y = Ax + b$, A 和 b 就是**参数**。

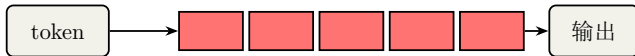
- ▶ 模型架构一确定, 参数的**规模**就固定了 (几百亿到上万亿);
- ▶ 训练时参数不断更新, 逐渐存下语言规律和知识;
- ▶ 推理 (回答你的问题) 时, 参数**不再改变**, 只是拿来做计算。

每处理 1 个 token, 都要让它“流过”这些参数做一遍运算。

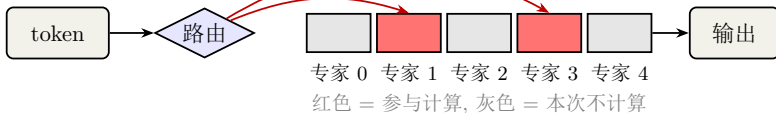
所以计算量 $\approx$ token 数量 $\times$ 参与计算的参数量。

两种模型: 全员上阵 vs. 只叫专家

Dense 模型: 全部参数都参与计算



~~MoE 模型: 只激活最相关的少数“专家”~~



Dense (稠密) 模型

推理时几乎所有参数都参与计算,资源消耗大

MoE (混合专家) 模型

先判断输入该交给哪几个“专家”，只激活这一小部分参数

这正是 DeepSeek 等模型能大幅降低使用成本的原因之一。

公开模型的参数对比

模型	总参数量	激活参数量	架构	最大上下文
GPT-3	175B	175B (全激活)	Dense	2K
LLaMA-2-70B	70B	70B (全激活)	Dense	4K
Qwen-72B	72B	72B (全激活)	Dense	32K
DeepSeek-R1	671B	37B (仅部分激活)	MoE	128K

B = Billion (十亿), 175B 即 1750 亿个参数。

DeepSeek-R1 总参数是 GPT-3 的近 4 倍, 但每个 token 实际只用 37B 参数计算——

总参数决定“会多少”, 激活参数决定“一次算多贵”。

按 token 计费的逻辑, 现在可以说清了

$$\text{计算成本} \approx \text{输入输出的 token 数} \times \text{被激活的参数规模}$$

- ▶ token 数既反映输入规模, 也间接体现实际算力消耗;
- ▶ 激活参数越少的模型, 单 token 成本越低, 所以定价也更便宜;
- ▶ 输出比输入贵, 因为输出是一个一个 token 生成的, 每生成一个都要完整算一遍。

Q: 现在再看开场那张定价表: 为什么 mini 版模型比旗舰版便宜十几倍?
为什么“缓存输入”更便宜?

成本意识: 用得起, 更要用得聪明

第一讲说过“Token 自费, 但不需要 Tokenmaxxing”。现在你知道钱花在哪了:

- ▶ 简单任务用小模型, 复杂任务再上大模型——别用旗舰模型改错别字;
- ▶ 长文档先做摘要再提问, 不要每次都把全文塞进上下文;
- ▶ 输出比输入贵, 让模型“只回答要点”往往省钱又省时间。

Q:讨论 (2 分钟): 你上周用 AI 的经历里, 哪一次是“大炮打蚊子”? 换成什么做法会更省?

基础零件讲完了, 接下来看四个把模型用起来的“工具”⇒ 第五部分

工具一: Prompt (提示词)

Prompt 是你使用大模型时, 提供给它的**输入内容或指令**。
它告诉模型: 你希望它完成什么任务, 以什么方式回答。

用户输入“帮我介绍一下大模型。”——这句话就是一个 prompt。

模型接到 prompt 后, 就按第一讲讲过的方式: 逐个 token 生成回答。

Prompt 能做三件事

引导行为 不同的提示词, 让模型生成不同风格、不同方向的内容。

控制质量 精心设计的 prompt, 能让回答更准确、更符合预期。

扩展功能 复杂的 prompt 可以包含任务说明、角色设定、格式要求, 甚至函数调用。

使用技巧

开源基础模型多是“通才”: 理解力和表达力强, 但不专注某个领域。想让它在化工、医学、法律等垂直领域输出高质量内容, 就要设计**更有针对性的 prompt**。

Prompt 看似简单, 其实没那么容易——第三讲整讲都讲它。

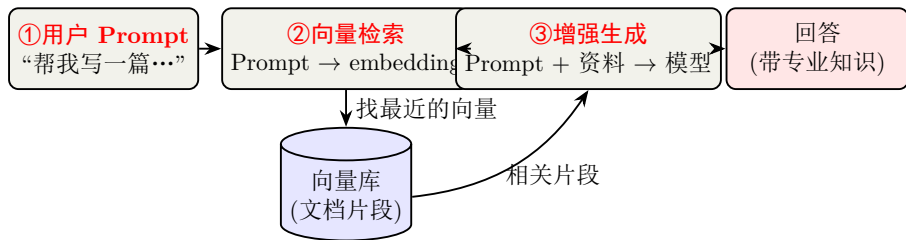
工具二: RAG (检索增强生成)

RAG = 大模型的**生成能力** + 外部知识的**检索能力**

在生成文本之前, 先去外部资料里**动态检索**相关信息,
让模型不只依赖自己训练时记住的参数, 还能**引用外部数据**作答。

简化理解: RAG 让模型有了“随时可以翻阅百科或文档”
的能力,
而不是只靠自己的记忆答题。

RAG 的工作流程: 三步



1. 用户输入 **Prompt**: 如“帮我写一篇关于机器人控制算法的科普文章”;
2. **向量检索**: 把 Prompt 转成 embedding, 到**向量库**里找最相关的文档片段——靠的就是第二部分讲的“向量距离”;
3. **增强生成**: 检索到的内容和原始 Prompt 一起送进模型, 生成带专业知识的回答。

化工同学的类比: 闭卷 vs. 开卷



化工设计手册: 工程师不背, 但
随时查

只靠参数: 闭卷考试, 全凭记忆, 记错了也不知道。

RAG: 开卷考试, 先翻手册找到那一页, 再作答。

RAG 特别适合:

- ▶ 知识**经常更新** (最新文献、内部规程);
- ▶ 知识**很专业、很小众**, 训练语料里几乎没有;
- ▶ 需要**给出依据**, 让人能核对来源。

工程师查手册不算作弊, 模型查资料也不算——**关键是查得准、答得有出处。**

案例分析：给实验室做一个“文献助手”

课堂互动⑤

实验室有 2000 篇重油加工相关的 PDF，老师希望能直接提问：“近三年关于渣油加氢催化剂失活的结论有哪些？”

4 人一组，用 RAG 的三步，写出你们的方案：每一步要做什么？用到今天哪些关键词？

提示问题：

- ▶ 2000 篇 PDF 怎么变成“向量库”？（切块、embedding、存储）
- ▶ 提问时检索出 20 段内容，全部塞给模型会不会超上下文？怎么取舍？
- ▶ 模型回答时，怎样让老师能点开原文核对？

这个案例在第四讲会真的做出来。

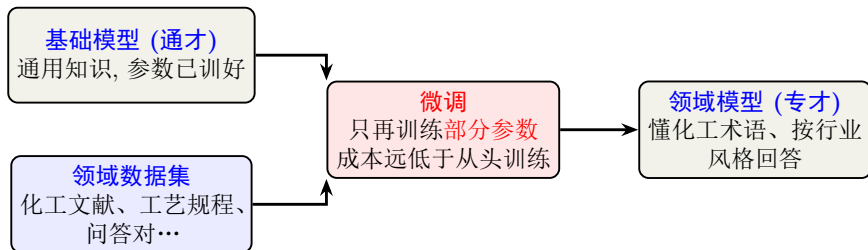
工具三：微调 (Fine-tuning) ——什么时候需要它？

通常我们会先用 **Prompt** 和 **RAG** 引导模型完成任务。
但有些场景下, 两者都用了, 效果仍达不到预期:

1. 模型对某些领域的**专业知识理解不够深入**;
2. 模型的**表达风格**与业务需求有差距;
3. 任务需要模型掌握**新的知识**, 而这些知识在训练语料里很少或根本没有。

这时, 就需要在现有大模型的基础上做**微调**。

微调是什么：在“通才”的基础上再上一门专业课



类比: 大学毕业生 (通才) + 岗前培训 (领域数据) → 工艺工程师 (专才)

核心思想: 模型已经学了大量通用知识 → 采集并构建一个**新的、垂直领域的训练数据集**

→ 对模型的**部分参数**再训练 → 让它学会新的模式、知识或任务。

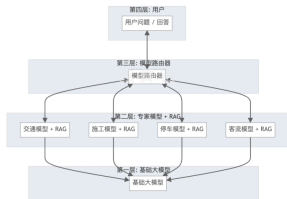
相比从头训练: **计算成本和数据需求低得多**, 又能显著提升特定场景的效果。

三个工具怎么选？一张对比表

	Prompt	RAG	微调
改变什么	输入	输入 (加检索到的资料)	参数
解决什么	说清需求、控制格式	知识不够新、不够专、要有依据	风格、深层专业能力、新任务
成本	几乎为零	中 (建向量库)	高 (数据 + 算力)
生效速度	立刻	小时级	天级

顺序：先 Prompt，再 RAG，最后才考虑微调。能用便宜的办法解决，就别上贵的。

工具四：模型路由——“大脑的分配中心”



实际系统里, 往往在一个基础模型之上, 微调出**多个领域专用模型**: 交通模型、施工模型、停车模型、客流模型
.....

用户的一个问题该交给谁? 需要一个**模型路由器**来调度。

它是纯软件逻辑, 不是硬件设备: 解析请求 → 判断归属 → 分配给合适的专家模型 → 返回结果。

模型路由器的四项职责

- 解析输入** 对用户请求做语义解析、分类、关键词提取。例：“火车站排队情况” → 客流 / 交通领域。
- 选择模型** 根据意图和**置信度**选专家模型；置信度低就回退到**通用大模型 + RAG**兜底，避免答非所问。
- 控制流转** 把请求转发给目标模型，并记录路由日志，便于追踪和优化。
- 结果融合** 需要多个模型协同的问题（如“地铁施工对火车站客流的影响”），调用多个模型并融合结果。

像极了医院的分诊台：先问清症状，再分到对应科室；拿不准的先去全科。

再往前一步：让大模型链接物理世界



炼厂中控室：成千上万个实时数据点

城市版：你问“附近哪里有空车位？周边堵不堵？”系统即时调用停车场数据、交通监测、视频流，再由大模型解释说明。

化工厂版：值班工程师问“三号加氢反应器今晚温升为什么偏快？”系统调用 DCS 实时数据、历史工况库 (RAG)、工艺专家模型 (路由)，给出解释和建议。

AI 不再只回答“文字上的问题”，而是成为人和真实世界之间的**智能接口**。

小组：为一座炼厂设计“AI 值班助手”

课堂互动⑥

4 人一组, 5 分钟。用今天的 7 个关键词各说一句话, 描述这个助手是怎么工作的:

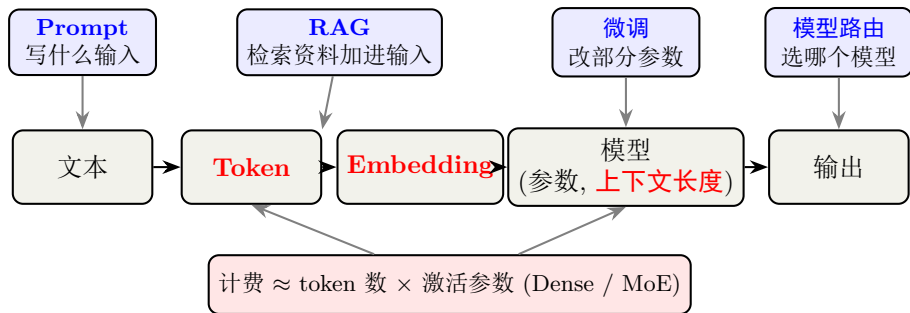
Token · Embedding · 上下文长度 · Prompt · RAG · 微调 · 模型路由

示例开头: “工程师的提问先被切成 token ……”

评价标准: 7 个词是否都用对了位置; 有没有说清“为什么这一步需要这个词”。

选 2 组分享, 其他组挑错——说得哪里用错了, 也算学会了。

本讲总结: 七个关键词, 一条流水线



文本 → token → embedding → 模型 (受上下文长度限制, 用参数计算) → 输出;

Prompt 决定输入什么, RAG 往输入里加资料, 微调改参数, 路由决定用哪个模型。

七个词, 各用一句话带走

Token	模型处理文本的最小单位, 计费 and 上下文都按它算
Embedding	把 token 变成向量, 意思相近则向量相近
上下文长度	一次能看多少 token, 超出就忘掉最前面
Prompt	给模型的输入和指令, 决定它做什么、怎么答
RAG	先检索外部资料再生成, 让模型“开卷答题”
微调	用领域数据再训练部分参数, 通才变专才
模型路由	判断问题该交给哪个专家模型, 拿不准就兜底

Q:回到开场: 现在你能解释“为什么按 token 收费”和“为什么会忘事”了吗?

随堂快测

课堂互动 · 3 道题

Q1 (单选) 下列哪个数字最可能是“Fluid Catalytic Cracking”被切出的 token 数?

A. 1 B. 3 ~ 5 C. 24 D. 100

Q2 (判断) RAG 会修改大模型的参数。

Q3 (简答) 一位同学抱怨: “我把整本教材贴给 AI, 它却只回答了最后一章的内容。” 用今天的词解释原因, 并给一个改进办法。

答案: Q1 选 **B**; Q2 **×** 错误, RAG 只改输入, 微调才改参数;
Q3 超出上下文长度被截断; 改进: 分章提问, 或用 RAG 只检索相关段落。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 动手测: 用在线 tokenizer (token4all.cn/token) 分别输入一段 200 字的中文和它的英文翻译, 记录 token 数, 解释差异 (150 字)。
- 必做② 算成本: 查任意一家大模型的 API 定价, 估算“让模型读完你本学期一门课的全部讲义并生成复习提纲”要花多少钱, 写出计算过程。
- 选做③ 设计: 把课堂互动⑥的“炼厂值班助手”整理成一页方案图, 标出 7 个关键词的位置。

拓展阅读: [DeepSeek 文档: Token 用量说明](#); [DeepSeek-V3 技术报告](#)中关于 MoE 的章节 (选读)。

下讲预告: Prompt 很简单? 其实没你想的那么容易

为什么早期 ChatGPT 总“答非所问”

Prompt Engineer 是干什么的

一条 Prompt 的四个部分

产品级 Prompt 为什么长达几百行

著名的“奶奶漏洞”

因为大家还不会写 Prompt

一度被叫做“AI 驯兽师”

角色、指令、输入、输出格式

安全、异常、多轮、工具调用

Prompt 不只是“我要什么”，还有“不能给什么”

Q:课前思考: 你有没有一次觉得 AI “没听懂”你的话? 把那条提问原样带来。

Thank
You!

 Your opinion
Matters

第二讲·走进大模型——你必须懂的关键词
周天航·中国石油大学(北京)

下课。请完成课后任务,下次课把你那条“AI 没听懂”的提问带来。